

Projektdokumentation

Game Gallery

Modul 294 – Leistungsbeurteilung

Name	Gabriell Prenaj / Damjan Panic
Klasse	UIFZ – 2524 - 021
Datum	06.06.2026
Lehrperson	Graziano Laveder

Inhaltsverzeichnis

1. Projektidee	3
2. Anforderungskatalog (User Stories)	3
3. Klassendiagramm	4
4. Storyboard.....	5
5. Screen-Mockups	5
6. REST-Schnittstellen.....	8
7. Testplan	11
8. Testergebnisse	12
9. Installationsanleitung	13
10. Hilfestellungen	13

1. Projektidee

Game Gallery ist eine Web-Applikation zum Verwalten der eigenen Videospiel-Sammlung. Spielerinnen und Spieler verlieren schnell den Überblick, welche Spiele sie besitzen, gespielt haben oder noch spielen wollen. Genau hier setzt die App an: Sie bietet einen einfachen, persönlichen Katalog der eigenen Games.

Jedes Spiel wird mit Titel, Plattform, Genre, einer Bewertung von 1 bis 5 Sternen, einem optionalen Cover-Bild und einer Notiz erfasst. Alle Spiele werden übersichtlich als Karten-Galerie angezeigt. Einträge können jederzeit hinzugefügt, bearbeitet oder gelöscht werden.

Technisch besteht die App aus einem React-Frontend, das über eine REST-API (json-server) mit einer JSON-Datenbank kommuniziert. Damit lassen sich alle Daten dauerhaft speichern und über die vier Grundoperationen (Anlegen, Lesen, Ändern, Löschen) verwalten.

2. Anforderungskatalog (User Stories)

User Story 1 – Spiele-Galerie ansehen

Als Nutzer möchte ich alle meine gespeicherten Spiele in einer Übersicht sehen, **damit** ich schnell einen Überblick über meine Sammlung bekomme.

Akzeptanzkriterien:

- Beim Öffnen der Startseite werden alle Spiele als Karten angezeigt.
- Jede Karte zeigt Titel, Plattform, Genre und Bewertung.
- Ist ein Cover-Bild hinterlegt, wird es angezeigt, sonst ein Platzhalter.
- Sind keine Spiele vorhanden, erscheint ein passender Hinweistext.

User Story 2 – Neues Spiel erfassen

Als Nutzer möchte ich ein neues Spiel mit allen Angaben erfassen können, **damit** ich meine Sammlung erweitern kann.

Akzeptanzkriterien:

- Über "Spiel hinzufügen" gelange ich zu einem Formular.
- Ich kann Titel, Plattform, Genre, Bewertung, Bild-Link und Notiz eingeben.
- Lasse ich den Titel leer, erscheint eine Fehlermeldung und es wird nicht gespeichert.
- Nach erfolgreichem Speichern erscheint das neue Spiel in der Galerie.

User Story 3 – Spiel bearbeiten und löschen

Als Nutzer möchte ich bestehende Spiele ändern oder entfernen können, **damit** meine Sammlung immer aktuell bleibt.

Akzeptanzkriterien:

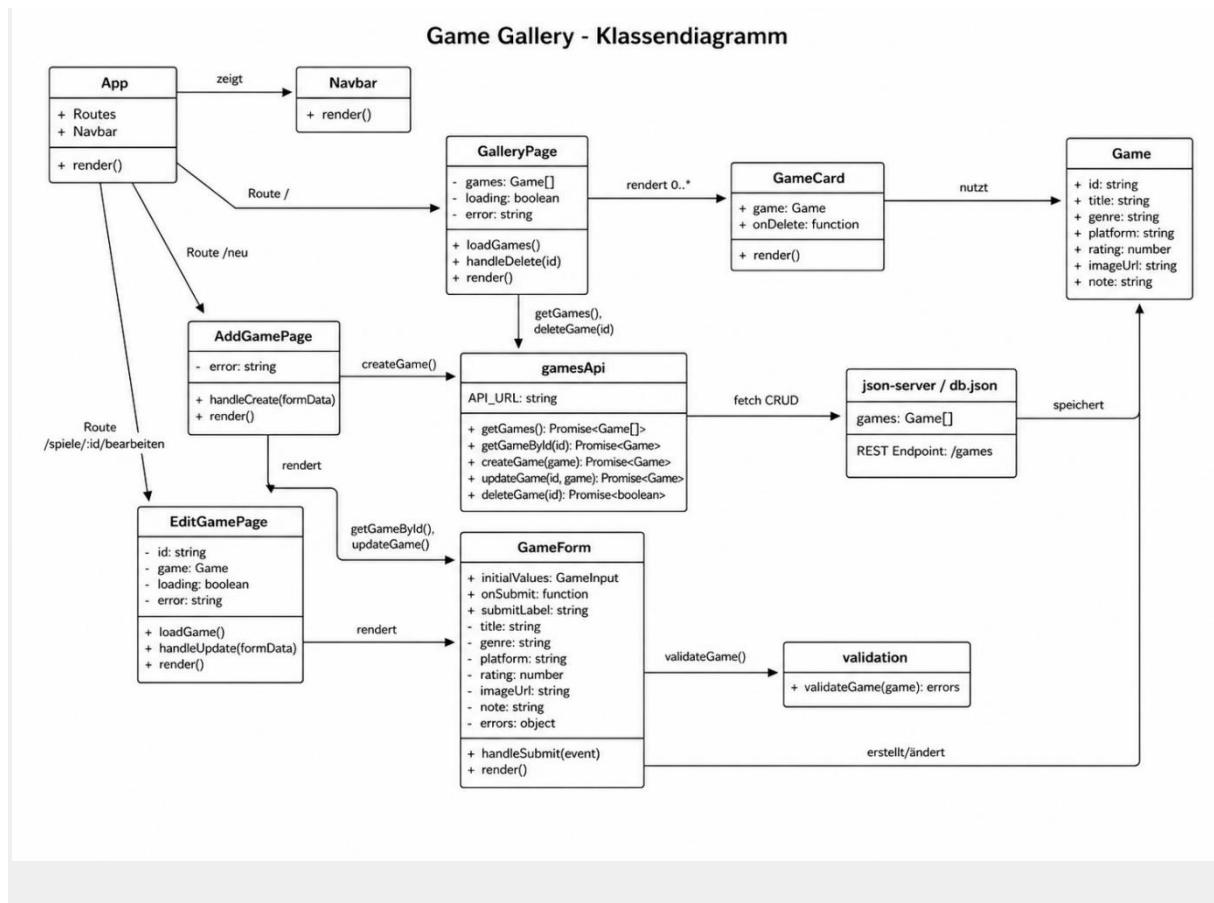
- Über "Bearbeiten" kann ich die Angaben eines Spiels anpassen und speichern.
- Über "Löschen" kann ich ein Spiel entfernen.
- Vor dem Löschen erscheint eine Sicherheitsabfrage.
- Gelöschte Spiele verschwinden sofort aus der Galerie.

Definition of Done (für alle User Stories)

- Alle Akzeptanzkriterien sind erfüllt.
- Der Code ist sauber strukturiert und nachvollziehbar.
- Die Funktion wurde manuell getestet (siehe Testplan).
- Es bestehen automatische Unit-Tests für die zentrale Logik.

3. Klassendiagramm

Die Datenstruktur basiert auf einem einzigen Datentyp Game. Da die JSON-Struktur flach ist, wird zusätzlich die konzeptuelle Klasse GameCollection ergänzt, die im Frontend die Liste aller Spiele verwaltet.

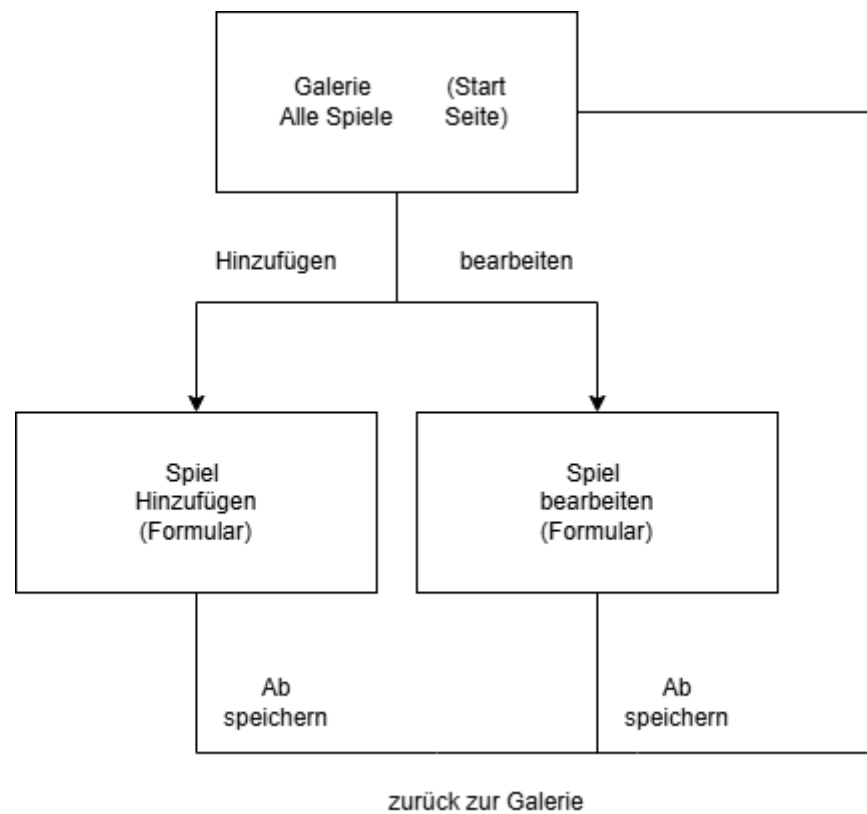


Was hier wichtig ist:

- Jede Klasse hat Attribute mit Datentyp (z.B. `+String title`, `+int rating`).
- Die Beziehung ist mit Kardinalität beschriftet: `GameCollection 1 : n Game`.
- `GameCollection` ist eine konzeptuelle Klasse, die die Liste im Frontend abbildet.

4. Storyboard

Das Storyboard zeigt den Navigationsfluss durch die App.

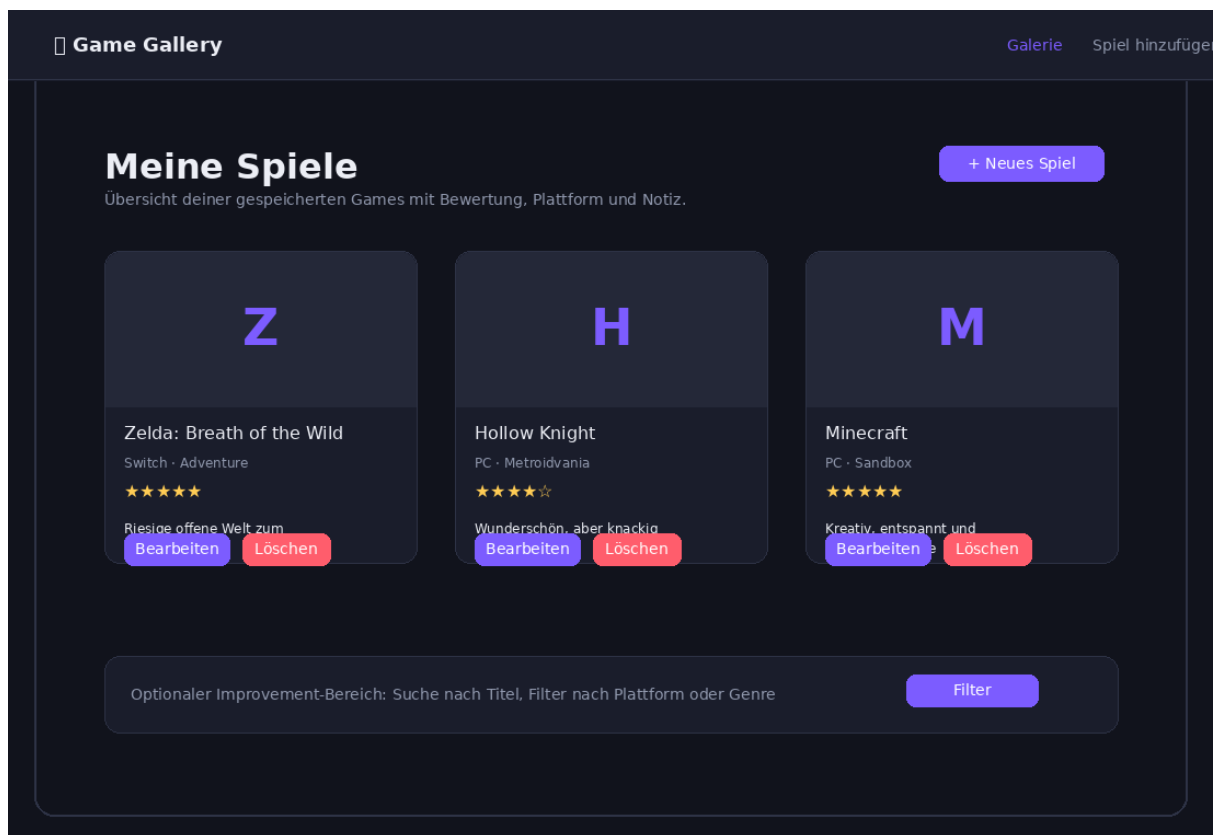


5. Screen-Mockups

Die App besteht aus drei Screens. Jeder Screen ist nachfolgend beschrieben; die Skizze wird als Bild eingefügt.

Screen 1 – Galerie (Startseite)

- Oben: Navigationsleiste mit Logo "Game Gallery" und Links "Galerie" und "Spiel hinzufügen".
- Überschrift "Meine Spiele".
- Raster aus Spiel-Karten mit Cover (oder Platzhalter), Titel, Plattform, Genre, Sternebewertung sowie Buttons "Bearbeiten" und "Löschen".



Screen 2 – Spiel hinzufügen

- Navigationsleiste wie oben, Überschrift "Neues Spiel hinzufügen".
- Formular mit Feldern: Titel*, Plattform, Genre, Bewertung (1–5)*, Bild-Link, Notiz.
- Button "Spiel hinzufügen". Bei leerem Titel: Fehlermeldung unter dem Feld.

Game Gallery

GalerieSpiel hinzufügen

Neues Spiel hinzufügen

Formular für CREATE: Titel und Bewertung sind Pflichtfelder.

Titel *

z.B. Minecraft

Plattform

z.B. PC, PS5, Switch

Genre

z.B. Adventure

Bewertung (1-5) *

3

Bild-Link (optional)

https://...

Notiz (optional)

Deine Meinung zum Spiel

Live-Vorschau

So könnte die Karte nach dem Speichern aussehen.

M

Minecraft

PC · Sandbox

★★★★☆

Deine Meinung zum Spiel

BearbeitenLöschen

Modul 294 – Game Gallery · Seite 7

Screen 3 – Spiel bearbeiten

- Wie Screen 2, aber mit vorausgefüllten Werten und Überschrift "Spiel bearbeiten".
- Button "Änderungen speichern".

Game Gallery

GalerieSpiel hinzufügen

Spiel bearbeiten

Update-Seite mit vorausgefüllten Werten und sichtbarer Validierung.

Titel *

Titel darf nicht leer sein.

Plattform

PC

Genre

Metroidvania

Bewertung (1-5) *

4

Bild-Link (optional)

https://...

Notiz (optional)

Wunderschön, aber knackig schwer.

Löschen bestätigen

Dieses Spiel wirklich löschen?

Der Eintrag wird aus der Galerie entfernt.

AbbrechenLöschen

Feedback-Idee

Nach dem Speichern könnte eine kleine Erfolgsmeldung e

Modul 294 – Game Gallery · Seite 8

6. REST-Schnittstellen

Für die Datenspeicherung wird json-server als REST-API verwendet. Die API liest und schreibt in die Datei db.json.

Basis-URL

```
http://localhost:3001
```

Im Frontend konfiguriert über die Umgebungsvariable VITE_API_URL (Standardwert: http://localhost:3001).

Datentyp: Game

```
{
  "id": "1",
  "title": "The Legend of Zelda: Breath of the Wild",
  "genre": "Adventure",
  "platform": "Switch",
  "rating": 5,
  "imageUrl": "",
  "note": "Riesige offene Welt zum Erkunden."
}
```

Feld	Typ	Beschreibung	Pflicht
id	String	Eindeutige ID, von json-server generiert	automatisch
title	String	Titel des Spiels	ja
genre	String	Genre, z.B. Adventure	nein
platform	String	Plattform, z.B. PC, PS5, Switch	nein
rating	int	Bewertung von 1 bis 5	ja
imageUrl	String	Link zum Cover-Bild	nein
note	String	Persönliche Notiz	nein

Endpoints

GET /games – Liefert alle Spiele

- Methode: GET
- Response: 200 OK, Array von Game-Objekten
- Fehler: 500 bei Server-Problem

GET /games/{id} – Liefert ein einzelnes Spiel

- Methode: GET
- Response: 200 OK, ein Game-Objekt
- Fehler: 404, falls die ID nicht existiert

POST /games – Erstellt ein neues Spiel

- Methode: POST, Header: Content-Type: application/json
- Body: Game-Objekt (ohne id)
- Response: 201 Created, das gespeicherte Game-Objekt inkl. neuer id

PUT /games/{id} – Aktualisiert ein bestehendes Spiel

- Methode: PUT, Header: Content-Type: application/json
- Body: komplettes Game-Objekt

- Response: 200 OK, das aktualisierte Objekt
- Fehler: 404, falls die ID nicht existiert

DELETE /games/{id} – Löscht ein Spiel

- Methode: DELETE
- Response: 200 OK
- Fehler: 404, falls die ID nicht existiert

7. Testplan

Der Testplan dokumentiert manuelle Tests, die nach der Implementierung durchgeführt werden.

T1: Galerie zeigt alle Spiele an

Vorbedingung: Mindestens ein Spiel ist in der Datenbank.

Schritte:

1. json-server und Frontend starten.
2. Startseite (Galerie) aufrufen.

Erwartetes Ergebnis: Alle gespeicherten Spiele werden als Karten angezeigt.

T2: Neues Spiel hinzufügen

Vorbedingung: App ist gestartet.

Schritte:

1. Auf "Spiel hinzufügen" klicken.
2. Titel, Plattform, Genre und Bewertung ausfüllen.
3. Auf "Spiel hinzufügen" klicken.

Erwartetes Ergebnis: Das neue Spiel erscheint in der Galerie und ist über die API abrufbar.

T3: Validierung bei leerem Titel

Vorbedingung: Formular "Spiel hinzufügen" ist geöffnet.

Schritte:

1. Feld "Titel" leer lassen.
2. Restliche Felder ausfüllen.
3. Auf "Spiel hinzufügen" klicken.

Erwartetes Ergebnis: Eine Fehlermeldung erscheint, das Spiel wird NICHT gespeichert.

T4: Spiel bearbeiten

Vorbedingung: Mindestens ein Spiel existiert.

Schritte:

1. Bei einem Spiel auf "Bearbeiten" klicken.
2. Bewertung ändern.
3. Auf "Änderungen speichern" klicken.

Erwartetes Ergebnis: Die geänderte Bewertung wird in der Galerie angezeigt.

T5: Spiel löschen mit Bestätigung

Vorbedingung: Mindestens ein Spiel existiert.

Schritte:

1. Bei einem Spiel auf "Löschen" klicken.
2. In der Sicherheitsabfrage bestätigen.

Erwartetes Ergebnis: Das Spiel verschwindet aus der Galerie und ist nicht mehr über die API abrufbar.

T6 (optional): Fehlerhandling bei gestopptem json-server

Vorbedingung: json-server ist gestoppt.

Schritte:

1. Frontend neu laden.
2. Galerie aufrufen.

Erwartetes Ergebnis: Eine verständliche Fehlermeldung wird angezeigt, kein leerer Bildschirm.

8. Testergebnisse

ID	Testfall	Status	Bemerkung
T1	Galerie zeigt Spiele an	OK	
T2	Neues Spiel hinzufügen	OK	
T3	Validierung bei leerem Titel	OK	
T4	Spiel bearbeiten	OK	
T5	Spiel löschen	OK	
T6	Fehlerhandling (Server gestoppt)	OK	

Automatische Unit-Tests

Zusätzlich existieren automatische Unit-Tests (ausführbar mit `npm test`). Aktueller Stand: 9 Tests, alle bestanden.

Test	Was wird geprüft
validation.test.js	Validierung meldet Fehler bei leerem Titel und ungültiger Bewertung; meldet keinen Fehler bei korrekter Eingabe
GameCard.test.jsx	Karte zeigt Titel sowie Plattform und Genre korrekt an
GameForm.test.jsx	Formular zeigt Fehler bei leerem Titel und speichert nicht; speichert bei korrekter Eingabe
gamesApi.test.js	API-Funktionen rufen die richtigen Endpoints mit der richtigen Methode auf

9. Installationsanleitung

Voraussetzungen

- Node.js (Version 18 oder höher) inkl. npm

Schritte

1. Projektordner entpacken bzw. das Repository klonen und in einer IDE (z.B. VS Code) öffnen.

Im Projektordner die Abhängigkeiten installieren:

```
npm install
```

Erstes Terminal – json-server (die REST-API) starten:

```
npm run server
```

Die API läuft danach auf <http://localhost:3001>.

Zweites Terminal – das Frontend starten:

```
npm run dev
```

Die App läuft danach auf localhost. Diese im Terminal gegebene Adresse im Browser öffnen oder durch klicken des Links im Terminal.

Hinweis: Beide Terminals müssen gleichzeitig laufen – json-server liefert die Daten, das Frontend zeigt sie an.

Tests ausführen:

```
npm test
```

10. Hilfestellungen

KI-Tools:

- Claude: Für das design technische HTML / CSS (UI UX) und Grammatik und design bei der Dokumentation
- Chat gpt: Hilfe beim Codieren (Stellen wo Ki benutzt wurde sind markiert)

Internet-Quellen:

- React-Doku (react.dev), json-server-Doku, Youtube videos:
<https://www.youtube.com/watch?v=oGO-DWTmKA>,
<https://www.youtube.com/watch?v=UYz-9UaUp2E&t=24s>,

Personen:

- -